DevBridge · B.Tech Project

Module 6 — The Verified Skill Graph

A complete guide, from scratch: what every file does, how evidence becomes a score, how scores decay, how the submodules talk to each other, and how to run all of it.

Covers	submodules 6.0 projection · 6.1 ontology · 6.2 ingestor · 6.3 scoring · 6.4 query · 6.5 integrity, plus the `m6` facade
Code version	all packages v1.0.0 · parameters 1.0.0 · projection rules 1.0.0 · integrity rules 1.0.0 · ontology 0.1.0
Verified	562 tests passing; demo, recovery gate and every CLI command run while writing this guide. Every number in the worked examples was recomputed from the code.
Repository	`~/BTP-ABC3`
Date	24 October 2026

How to read this guide

If you are new to the module, read chapters 1–3 in order: they give you the vocabulary and the overall picture. Chapter 4 maps every file. Chapters 5–12 go submodule by submodule, and chapters 8–10 are the mathematical heart: **how a score is computed, how it decays, and how it spreads across related skills**, each with a fully worked numeric example. Chapter 13 is the hands-on part (how to run everything and read the output).

Code identifiers appear in `monospace`. File paths are relative to the repository root.

Contents

1. What Module 6 is . . . 5

1.1 The problem it solves . . . 5

1.2 What the output looks like . . . 5

1.3 The ground rules (hard constraints) . . . 5

1.4 The six submodules at a glance . . . 6

2. Concepts you need first . . . 7

2.1 Data concepts . . . 7

2.2 The three numbers on every event . . . 7

2.3 Modelling concepts . . . 7

3. Architecture and how the submodules interact . . . 9

3.1 The one-way pipeline . . . 9

3.2 Narrow ports between submodules . . . 10

3.3 A request, end to end . . . 10

3.4 The contracts: shared truth . . . 11

4. File structure and the use of each file . . . 12

4.1 contracts/ . . . 12

4.2 packages/m6_projection — 6.0 . . . 13

4.3 packages/m6_ontology — 6.1 . . . 13

4.4 packages/m6_ingestor — 6.2 . . . 13

4.5 packages/m6_scoring — 6.3 . . . 13

4.6 packages/m6_query — 6.4 . . . 14

4.7 packages/m6_integrity — 6.5, and packages/m6 — the facade . . . 14

4.8 demo/, tests/, docs/ . . . 14

5. 6.0 — Event projection: from a report to evidence events . . . 16

5.1 What it reads from the report . . . 16

5.2 Step 0: the consent gate . . . 16

5.3 The seven event kinds and when each is emitted . . . 16

5.4 Repository substance bands and strength . . . 17

5.5 Skill relevance from languages and frameworks . . . 17

5.6 Reliability factors: every discount is named . . . 17

5.7 Dates: occurred_at is when it happened . . . 18

5.8 Deterministic event ids . . . 18

5.9 Suppressions and invariants . . . 18

6. 6.1 — Ontology: which skills exist and how they relate . . . 20

6.1 Contents of the seed . . . 20

6.2 Validation: a broken seed is refused . . . 20

6.3 Who uses which method . . . 20

7. 6.2 — Evidence ingestor: the gatekeeper and the archive . . . 21
7.1 The ingest decision, step by step . . . 21
7.2 Storage: three tables and four triggers . . . 21
7.3 Quarantine: held, not dropped . . . 21
7.4 Concurrency . . . 22
8. 6.3 — Scoring: how a score is computed . . . 23
8.1 The belief: a Gaussian in logit space . . . 23
8.2 The prior: below median, and wide . . . 23
8.3 Step 1 of each event: the observed value . . . 23
8.4 Step 2 of each event: the observation variance . . . 24
8.5 Step 3: the Kalman update . . . 24
8.6 Step 4: the fold, recomputing from the log . . . 25
8.7 Step 5: from state to the numbers you see . . . 25
8.8 Worked example 1: go, step by step . . . 26
8.9 Worked example 2: python, and why a weak claim barely matters . . . 26
8.10 Worked example 3: rust, a lone self-report . . . 27
9. Decay: how scores fade when evidence gets old . . . 28
9.1 The two effects . . . 28
9.2 The three tiers . . . 28
9.3 Worked decay examples . . . 29
9.4 Where decay happens . . . 29
10. Propagation: scores for skills with no direct evidence . . . 30
10.1 The mechanism: damping is variance inflation . . . 30
10.2 The presentation problem, and how it is handled . . . 30
11. 6.4 — Query API: skills, explanations and search . . . 31
11.1 Endpoints . . . 31
11.2 The explain response (the demo endpoint) . . . 31
11.3 Search: tiers, never a ranking . . . 32
12. 6.5 — Integrity analytics: what a careful reader should notice . . . 33
13. How to run it and get the output . . . 34
13.1 Prerequisites . . . 34
13.2 The fastest path: three commands . . . 34
13.3 All Make targets . . . 34
13.4 Step by step, one submodule at a time . . . 35
13.5 Running the HTTP services . . . 35
13.6 Using it from Python . . . 35
13.7 Running on your own report . . . 36
13.8 Reading the demo output . . . 36
14. How we know it works . . . 37

14.1 The test suite . . . 37
14.2 The parameter-recovery gate . . . 37
14.3 The sensitivity sweep . . . 37
15. Caveats and gotchas . . . 38
16. Appendix: formula sheet and parameter reference . . . 39
16.1 Every formula in one place . . . 39
16.2 m6-parameters.yaml, v1.0.0 . . . 39
16.3 Projection priors (projection-rules.yaml) . . . 40
16.4 Where to read more . . . 40

1. What Module 6 is

1.1 The problem it solves

DevBridge's Module 5 (M5) is a CV verification engine. It reads a candidate's CV, extracts every claim, and checks those claims against outside sources: GitHub repositories, competitive-programming platforms such as Codeforces, live deployments, and credential issuers. It outputs a **VerificationReport**, a large JSON document that says what was claimed, what evidence was gathered and what was found.

That report answers *"is this CV true?"*. Module 6 (M6) answers a different question: *"given everything we have verified, what can this person actually do, how sure are we, and how will that change as the evidence gets older?"*

M6 turns verified artefacts into a **skill graph**. That is one estimate per skill (TypeScript, algorithms, Docker…), and each estimate is an **interval**, not a single number. Every estimate can be taken apart event by event, so anyone can see why it has the value it has.

1.2 What the output looks like

This is the real output of the demo for one candidate (`subj_c4f80a12`), as of 30 August 2026:

skill	interval [lower, upper]	mean	evidence	basis
`algorithms`	`[0.703, 0.811]`	`0.761`	2	direct
`problem_solving`	`[0.692, 0.804]`	`0.752`	3	direct
`data_structures`	`[0.611, 0.792]`	`0.710`	0	inferred from algorithms
`cpp`	`[0.556, 0.708]`	`0.635`	3	direct
`go`	`[0.536, 0.700]`	`0.622`	2	direct
…	…	…	…	…
`rust`	`[0.130, 0.487]`	`0.273`	1	direct (a lone CV claim)

Read each row like this. The *mean* is the best single guess at ability on a 0–1 scale. The *interval* shows how uncertain that guess is. The published **score is the lower bound** of the interval. That choice is deliberate: a candidate with little evidence gets a wide interval and therefore a low score, even if their mean is high. Uncertainty costs you, and silence is never rewarded.

1.3 The ground rules (hard constraints)

These were agreed early in the design. They are enforced in code and tests, not just written down, and they explain many decisions that might otherwise look strange:

- **No trained models.** No machine learning, no optimiser, no labelled data. Every update is a closed-form formula, and every constant is set by documented reasoning and then stress-tested.
- **No overall score.** No single number sums up a person. `GET /developers/{ref}/score` deliberately returns HTTP 400 with an explanation.
- **Intervals, not rankings.** Two candidates may be ordered only when their intervals do not overlap. Search returns *tiers* of indistinguishable candidates.
- **The evidence log is append-only.** Scores are never stored as truth. They are recomputed by *folding* the immutable log, which makes time-travel (`as_of`), ablation and parameter

sweeps cheap.

- **Constants live in contracts, not code.** Every number, threshold, tier and edge is in a YAML file under `contracts/`, so you can change model behaviour without editing Python.
- **Consent is mandatory.** Nothing enters the skill graph without `skill_graph_inclusion` consent. This is checked twice.
- **Silence is never an option.** Every claim is either turned into an event or recorded in `not_emitted[]` with a reason.
- **Integrity findings never change a score.** Suspicious patterns are reported next to the graph, never folded into it.

1.4 The six submodules at a glance

#	package	one-line job	analogy
6.0	`m6_projection`	Turn an M5 report into a batch of *evidence events*	translator
6.1	`m6_ontology`	Say which skills exist, how fast each decays, what implies what	dictionary
6.2	`m6_ingestor`	Validate, consent-check, de-duplicate and store events forever	gatekeeper + archive
6.3	`m6_scoring`	Fold events into a belief per skill: Kalman update, decay, propagation	the maths
6.4	`m6_query`	Serve skills, explanations and tiered search over HTTP	front desk
6.5	`m6_integrity`	Point out notable patterns in the evidence, changing nothing	careful reader
—	`m6`	Facade: all six behind one Python object and one HTTP server	switchboard

2. Concepts you need first

Most of the module is easy once these terms are clear. Each is used constantly later.

2.1 Data concepts

term	meaning
subject_ref	An opaque id for the candidate, e.g. `subj_c4f80a12`. M6 never sees a name.
verification_id	Id of the M5 report every event came from, e.g. `ver_01K2DEMO7QX4N`.
claim	One thing the CV asserts: a skill, a project, an achievement, a job, a degree… Claims have ids like `c11` and a `kind`.
evidence bundle	What M5 gathered from the outside world: GitHub repos, platform ratings, deployment probes, credentials, and gaps (what could not be checked).
evidence event	M6's unit of input: one piece of capability evidence about one or more skills. It has a `kind`, a date (`occurred_at`), `skill_refs` with relevances, and `strength`, `reliability` and `direction`.
batch	All events from one report, plus `consent_scope` and `not_emitted[]`.
not_emitted / suppression	A record of something deliberately *not* turned into an event, with one of five fixed reasons.
quarantine	An event naming a skill the ontology does not know. It is stored but held out of scoring for that skill.
ontology	The list of known skills, their volatility tier, and the edges between them.
tier	`fundamental`, `ecosystem` or `framework`. The tier sets how fast evidence goes stale.

2.2 The three numbers on every event

Each event carries three numbers between 0 and 1. Keep them apart, because they play different roles:

field	question it answers	example
strength	How much signal does this *kind* of artefact carry about ability?	a large repo 0.60; a bare CV claim 0.30
reliability	How far do we trust the *source*, after all named discounts?	Codeforces API 0.90; scraped CodeChef 0.63; CV text 0.20
relevance	(per skill) How much of this observation is about *this particular skill*?	Go is 95% of a repo's bytes → 0.95

All three end up in one place, the observation variance, and together they decide how far an event moves the estimate (chapter 8).

2.3 Modelling concepts

term	meaning
belief / state	For each (candidate, skill) pair, M6 holds a Gaussian belief: a centre μ and a variance σ². It lives in *logit space* (below).
logit / sigmoid	logit(p) = ln(p / (1 − p)) maps (0, 1) onto the whole real line; sigmoid(x) = 1 / (1 + exp(−x)) maps it back. Maths happens in logit space; humans see display space.
prior	The belief before any evidence: centred at 0.25 (below median, on purpose), very wide.
observed value	What one event says the ability *is* (display space), e.g. 0.66 for a large repo.

term	meaning
observation variance	How noisy that statement is: $\sigma^2_{obs} = S0 / (\text{reliability} \times \text{relevance} \times \text{strength})$.
Kalman update	The closed-form rule for combining the belief with one observation. It produces a new centre and a smaller variance.
gain k	The fraction of the way the estimate moves towards an observation (0 to 1).
fold	Replaying every event for a skill in date order through the update rule, starting from the prior.
decay	Between events and up to the query date, variance grows and the centre drifts back to the prior. We *know less*, so the score falls.
propagation	A skill with no direct evidence can borrow a damped estimate from a neighbour one edge away (e.g. `c` from `cpp`).
as_of	The date you ask about. Each answer is the fold up to that date, decayed to that date.

3. Architecture and how the submodules interact

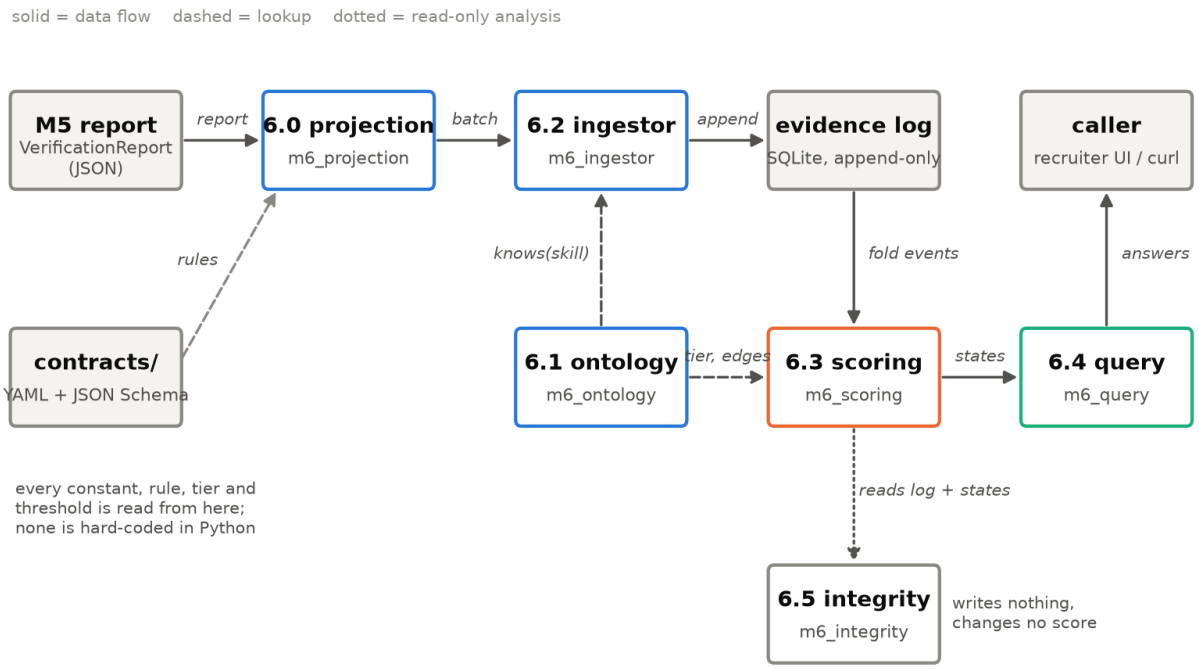


Figure 1 — The wiring of Module 6. Data flows one way, left to right and then down. 6.1 is consulted but never written to. 6.5 reads and never writes.

3.1 The one-way pipeline

```
report --6.0--> events --6.2--> log --6.3--> states --6.4--> answers
                                 |                            ^
                               6.1 ontology ------------------+
                                 |
                               6.5 reads the log and the states, writes nothing
```

Dependencies point in one direction only. Nothing downstream can change anything upstream, and the one deliberate exception is that 6.5 reads what 6.3 produces while nothing reads 6.5. Concretely:

submodule	reads	writes / returns	imports from
6.0 projection	an M5 report (dict); `projection-rules.yaml`	an `EvidenceEventBatch` (in memory)	nothing in M6
6.1 ontology	`ontology-seed.yaml`	answers: `knows`, `tier`, `relations`, `neighbours`	nothing in M6
6.2 ingestor	a batch; the event JSON Schema; the ontology port	rows in SQLite; a `Receipt`	6.1 (`SkillResolver`, `default_ontology`)
6.3 scoring	events (dicts); `m6-parameters.yaml`; `reference-distributions.yaml`; ontology	`SkillState` objects (never stored as truth)	nothing required; 6.1 / 6.2 only in its CLI
6.4 query	the `EventLog` (6.2); `ScoringEngine` (6.3); ontology (6.1)	JSON responses	6.3, and 6.2's log object

submodule	reads	writes / returns	imports from
6.5 integrity	events, states, quarantine list; `integrity-rules.yaml`	an `IntegrityReport`	nothing (engine passed in for half-lives)
`m6` facade	everything	convenience methods + combined HTTP	all six

3.2 Narrow ports between submodules

The interfaces between submodules are kept deliberately small, so that a responsibility cannot leak into the wrong place:

- **6.2 sees the ontology through a two-method port**, `SkillResolver` = `knows(skill_id)` + `ontology_version`. If the ingestor could reach the edges it might start traversing them, and propagation would move from read time (6.3, where it can be explained) to write time (where it cannot). A test runs the ingestor against a resolver that raises on any other attribute.
- **6.3 asks the ontology for** `tier(skill)` **and** `relations()` only. The tier decides the decay rate; the relations feed propagation.
- **6.4 reads the log through** `events_for_skill()` **and** `resolved_skills()`. These exclude quarantined skills, so the ingestor's quarantine decision also holds on the read path.
- **6.0 and 6.3 meet only through the event schema.** The projection emits *facts* in `payload` (e.g. `substance_band: large`); the scoring engine turns facts into numbers using `m6-parameters.yaml`. The projection decides what a repository *is*; the parameter file decides what it is *worth*.

3.3 A request, end to end

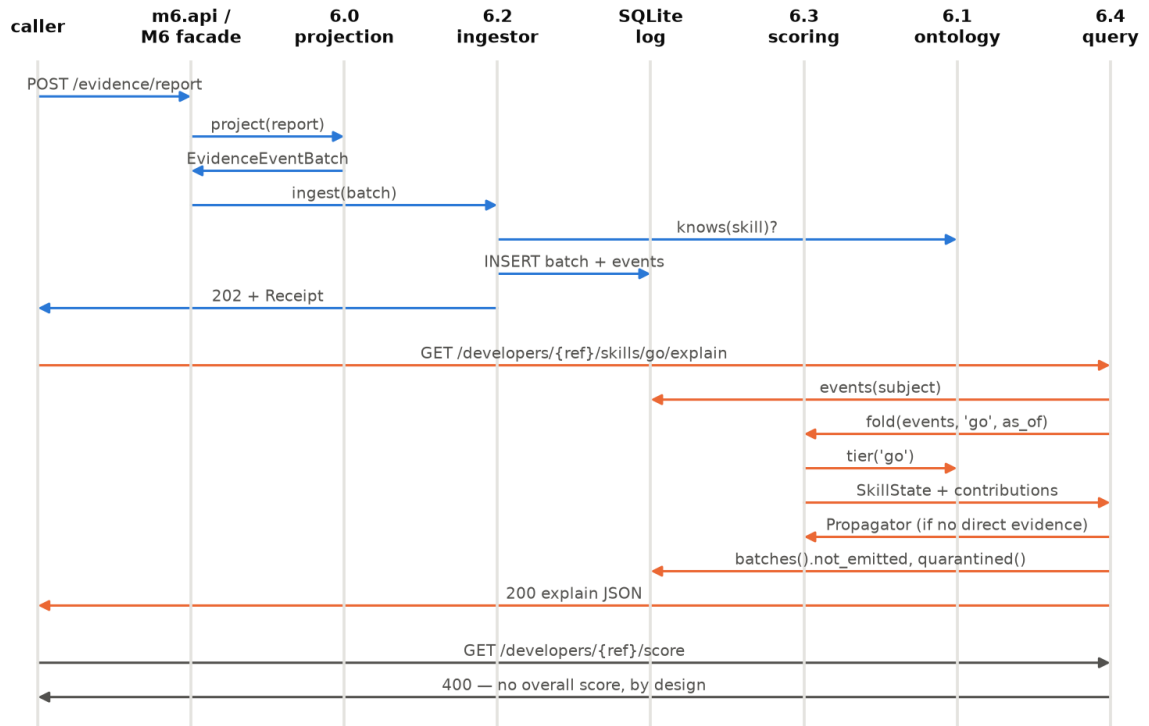


Figure 2 — Sequence for the write path (blue: posting a report) and the read path (orange: asking why `go` has its score), plus the deliberate refusal at the bottom.

Write path: `POST /evidence/report`

1. The combined server (`m6.api`) hands the raw M5 report to `M6.ingest_report()`.
2. 6.0 `project(report)` checks consent and walks the report, building events and suppressions. It enforces its invariants and returns a batch.
3. 6.2 `Ingestor.ingest(batch)` validates the batch against the JSON Schema, checks consent again, then takes an exclusive lock and decides: replay, conflict, or new.
4. For each new event it asks 6.1 `knows(skill)` for every skill named and flags unknown ones as quarantined.
5. It inserts the batch row, the event rows and one `event_skill` row per (event, skill) in one transaction, and returns `202` with a `Receipt` (accepted / duplicates / quarantined).

Read path: `GET /developers/{ref}/skills/{skill}/explain`

1. 6.4 `QueryService.explain()` loads the subject's events from the log.
2. It calls 6.3 `ScoringEngine.fold(events, skill, as_of)`. The engine asks 6.1 for the skill's tier, sorts the events, and replays them through decay and the Kalman update. It records a `Contribution` for each event.
3. If the skill has no direct evidence, it runs the `Propagator` over all direct states and the ontology edges.
4. It adds the interval, the basis, and `not_counted`: every suppression from the subject's batches plus any quarantined event naming this skill.
5. It returns 200 with the explain JSON.

Note that **no score is ever written anywhere**. Each read recomputes from the log. With tens of events per skill this takes microseconds.

3.4 The contracts: shared truth

Six machine-readable files fix all behaviour. Each has a single owner that reads it, and tests check that files which overlap agree with each other.

file	fixes	read by
`contracts/projection-rules.yaml`	what becomes an event; strengths, base reliabilities, named discounts, bands, aliases, suppressions, invariants	6.0
`contracts/ontology-seed.yaml`	27 skills, their tiers, 12 edges	6.1 (and through it 6.2, 6.3, 6.4)
`contracts/m6-parameters.yaml`	every constant in the scoring model, each with a sweep range and a rationale	6.3
`contracts/reference-distributions.yaml`	Codeforces rating → percentile table (ECDF)	6.3
`contracts/integrity-rules.yaml`	the nine integrity checks and their thresholds	6.5
`docs/docs/schemas/evidence-event.schema.json`	the exact shape of a batch and an event	6.2 (and 6.0's tests)

4. File structure and the use of each file

```
BTP-ABC3/
├── Makefile                      every run target (test, demo, project, ingest, score, ...)
├── contracts/                    machine-readable truth (YAML + JSON Schema)
│   ├── m6-parameters.yaml        scoring constants, sweep ranges, rationales
│   ├── projection-rules.yaml     report -> events rules
│   ├── ontology-seed.yaml        skills, tiers, edges
│   ├── integrity-rules.yaml      the nine checks
│   ├── reference-distributions.yaml   Codeforces ECDF anchors
│   ├── schemas/skill-state.schema.json  shape of a persisted SkillState
│   └── reference/
│       ├── m6_model.py           ~120-line executable SPEC of the model
│       └── verify_worked_example.py   re-checks the spec against the docs
├── packages/                     the implementation (put on PYTHONPATH)
│   ├── m6_projection/   6.0      params.py rules.py projector.py models.py errors.py __main__.py
│   ├── m6_ontology/     6.1      graph.py resolver.py errors.py __main__.py
│   ├── m6_ingestor/     6.2      ingestor.py store.py api.py errors.py __main__.py
│   ├── m6_scoring/      6.3      params.py observation.py filter.py engine.py
│   │                             propagation.py recovery.py errors.py __main__.py
│   ├── m6_query/        6.4      service.py ranking.py api.py errors.py __main__.py
│   ├── m6_integrity/    6.5      analyser.py findings.py rules.py
│   └── m6/                       facade.py api.py
├── demo/
│   ├── run_demo.py               the narrated end-to-end walkthrough
│   ├── input/verification-report-extensive.json   a messy, realistic M5 report
│   └── output/01..05-*.json      regenerated on every run
├── tests/                        562 tests; run.py is a zero-dependency runner
│   └── m6_projection/ m6_ontology/ m6_ingestor/ m6_scoring/ m6_query/ m6_integrity/ m6_whole/
└── docs/
    ├── docs/                     design documents (HTML), schemas/, examples/
    └── reports/                  LaTeX/PDF reports (this guide lives here too)
```

4.1 contracts/

file	purpose
`m6-parameters.yaml`	The **only place a scoring constant may live**: prior θ0 and σ0, S0, observed values per event kind, decay rates per tier, the half-life ratio k, propagation γ per edge type, display z. Each has a `value` (used by the engine) and a `sweep` range (used by the sensitivity sweep). `contract_version` becomes `parameter_version` on every state.
`projection-rules.yaml`	Emission rules per event kind, reliability factors (named discounts), suppressions, invariants, and `thresholds:` (substance bands, relevance formula, aliases, consent requirement). Changing it changes 6.0 with no code edit.
`ontology-seed.yaml`	Skills with labels and tiers; directed, typed edges. Growing the ontology is a pull request against this file.
`integrity-rules.yaml`	The nine checks with severity, thresholds and the mandatory `innocent_explanation`. Deleting a check switches it off.
`reference-distributions.yaml`	The Codeforces rated-population ECDF anchors used for `platform_rating`. Flagged `approximated_pending_regeneration`.
`schemas/skill-state.schema.json`	The persisted (μ, σ^2) state: a derived cache, not a source of truth.
`reference/m6_model.py`	A dependency-free executable specification. The real engine must reproduce its numbers (tested to 1e-9). If prose and this file disagree, this file wins.
`reference/verify_worked_example.py`	Re-checks the spec against the published worked example (`make verify`).
`README.md`	Rules for editing contracts (a parameter change invalidates stored states, nothing is fitted).

4.2 packages/m6_projection — 6.0

file	purpose
`projector.py`	`project(report)` walks the report section by section (platforms, repos, skill claims, achievements, deployments, credentials, uncorroborated certifications and projects, standing suppressions), builds events, sorts them, and runs `_check_invariants`.
`rules.py`	Pure decision functions with no I/O: `substance_band`, `repo_strength`, `language_relevances`, `framework_relevance`, `apply_factors`, `normalise_skill`, `event_id`.
`params.py`	Loads `projection-rules.yaml` into a `Rules` object. No threshold is hard-coded in the package.
`models.py`	Dataclasses mirroring the schema (`EvidenceEvent`, `SkillRef`, `ReliabilityFactor`, `EventSource`, `Suppressed`, `EvidenceEventBatch`), each with a schema-conformant `to_dict()`.
`errors.py`	`MalformedReport`, `ContractViolation`.
`__main__.py`	CLI: `python -m m6_projection report.json --out batch.json`.

4.3 packages/m6_ontology — 6.1

file	purpose
`graph.py`	`SeedOntology` (load, validate, `knows`, `tier`, `relations`, `neighbours`, `skills`, `skill_ids`), the `Skill` and `Edge` dataclasses, and `default_ontology()`.
`resolver.py`	The narrow `SkillResolver` protocol, plus `AcceptAll`, which resolves everything and reports version `accept-all`.
`errors.py`	`InvalidOntology` (carries *all* problems), `UnknownSkill`.
`__main__.py`	CLI: `check`, `skills [--tier]`, `edges`, `show <skill>`, `dump`.

4.4 packages/m6_ingestor — 6.2

file	purpose
`ingestor.py`	`Ingestor.ingest(batch)`: validate → consent → exclusive decision (replay / conflict / new) → per-event duplicate check and skill resolution → append. Returns a `Receipt`.
`store.py`	`EventLog`: SQLite schema (3 tables, indexes, **append-only triggers**), `content_hash`, read methods (`events`, `events_for_skill`, `resolved_skills`, `quarantined`, `batches`) and `append`. One re-entrant lock serialises all access.
`api.py`	HTTP server: `POST /evidence/batch`, `GET /health`, `/events`, `/quarantine`, `/batches`.
`errors.py`	Error types that *are* status codes: `SchemaInvalid` 400, `ConsentRefused` 403, `ConflictingEvent` 409, `ImmutableLog` 409.
`__main__.py`	CLI: `ingest batch.json --db x.db` and `serve --port 8100`.

4.5 packages/m6_scoring — 6.3

file	purpose
`params.py`	`Parameters`: typed accessors over `m6-parameters.yaml` (θ0, σ0, S0, λ per tier, k, γ, z, ECDF), `half_life(tier)`, `replacing(**overrides)` for sweeps, and `sweepable()`, which discovers every swept constant by walking the YAML.
`observation.py`	`ObservationModel`: event → (observed value, variance). Four methods: flat constant, substance band, proficiency band, percentile against reference. Raises `UnscoreableEvent` rather than guessing.

file	purpose
`filter.py`	`logit`, `sigmoid`, the immutable `State(mu, var, at)` with `mean`, `score`, `upper`, `interval`, `band`; and `Filter` with `initial`, `decayed_to`, `updated`, `gain`.
`engine.py`	`ScoringEngine`: `fold`, `fold_all`, `fold_without` (ablation), `rehydrate` (from cache), `explain`, `tier_of`. Defines `SkillState` and `Contribution`.
`propagation.py`	`Propagator` (read-time, one hop, direct sources only) and `edges_from_ontology`.
`recovery.py`	Parameter-recovery simulation: the gate that proves the filter can recover a known ability from synthetic evidence.
`errors.py`	`ScoringError`, `UnscoreableEvent`, `ParameterMismatch`.
`__main__.py`	CLI: `score`, `explain --skill`, `recover`, `sweep`.

4.6 packages/m6_query — 6.4

file	purpose
`service.py`	`QueryService`: `subjects`, `skills` (the graph as `SkillView` rows), `explain` (with `not_counted`), `search` (tiers), `known_skills`.
`ranking.py`	`Candidate`, `Tier`, `meets_threshold(definite=…)` and `tiers()`: the partial order by non-overlapping intervals.
`api.py`	Routes and HTTP handler, including the refusal route for `score|rating|trust|overall`.
`errors.py`	`QueryError`, `UnknownSubject`, `UnknownSkill`, `AggregateRefused` (400).
`__main__.py`	CLI: `skills`, `explain`, `search`, `serve`.

4.7 packages/m6_integrity — 6.5, and packages/m6 — the facade

file	purpose
`m6_integrity/analyser.py`	`IntegrityAnalyser.analyse()`: four whole-graph checks, four per-skill checks, `INFERRED_ONLY` over states; sorted and capped.
`m6_integrity/findings.py`	`Finding` and `IntegrityReport` dataclasses; the two severities.
`m6_integrity/rules.py`	Loads `integrity-rules.yaml`: `enabled`, `param`, `severity`, `innocent_explanation`.
`m6/facade.py`	`M6`: constructs all six in the right order and exposes `project`, `ingest_report`, `ingest_batch`, `skills`, `explain`, `search`, `states`, `integrity`, `profile`, `versions`.
`m6/api.py`	One HTTP server for everything. It reuses 6.4's routes verbatim and adds `/evidence/report`, `/evidence/batch`, `/versions`, `/developers/{ref}/integrity` and `/profile`.

4.8 demo/, tests/, docs/

path	purpose
`demo/run_demo.py`	Narrated, 9-step walkthrough of every submodule on one report; writes `demo/output/`.
`demo/input/verification-report-extensive.json`	30 claims, 7 repos, 2 platforms, deliberately messy.
`demo/output/01-projected-batch.json`	What 6.0 emitted and what it refused to emit.
`demo/output/02-ingest-receipt.json`	The ingest receipt: accepted / duplicates / quarantined.
`demo/output/03-skill-graph.json`	Every skill as an interval (plus the two-years-later graph).
`demo/output/04-explain.json`	One skill fully decomposed.

path	purpose
`demo/output/05-integrity.json`	The integrity findings.
`tests/run.py`	Zero-dependency runner (`make test`, or `make test FILTER=substring`).
`tests/m6_*/`	One folder per submodule, plus `m6_whole` for composition, HTTP and the demo.
`docs/docs/*.html`	Design documents: `m6-decisions.html` (decisions O1–O7), `m6-skill-graph-plan.html`, `m6-worked-trace.html`, `cv-to-skill-graph.html`, M5 specs.
`docs/docs/examples/`	`report-full.json` (the golden report), `report-degraded.json` (no consent), `evidence-events.json`.
`docs/docs/schemas/`	`evidence-event.schema.json` (M6's input contract), `verification-report.schema.json` (M5's output).

5. 6.0 — Event projection: from a report to evidence events

The projection is the seam between M5 and the skill graph. It is the *only* place that decides what capability signal a verified artefact carries. It does not decide whether a claim is true (M5 did that) and it does not compute a score (6.3 does that). It is deterministic: the same report always produces a byte-identical batch.

5.1 What it reads from the report

Top-level keys used: `verification_id`, `subject` (`subject_ref`, `consent_ref`), `document.created_at`, `valid_as_of`, `claim_graph.claims[]`, and `evidence_bundle` with `github` (ownership + repos), `platforms`, `deployments`, `credentials` and `gathered_at`.

5.2 Step 0: the consent gate

Before reading any evidence, the projector checks `subject.consent_ref`. If it is missing, it returns a batch with **zero events** and one suppression: “every capability signal in this report” with reason `consent_scope_excluded`. Otherwise the batch's `consent_scope` becomes `[skill_graph_inclusion, recruiter_visibility]`.

5.3 The seven event kinds and when each is emitted

kind	emitted when	strength	rel. base	skills (relevance)	dated at
`platform_rating`	platform found and has a max rating	0.80	0.90	algorithms 0.95, problem_solving 0.90, + skills of linked achievement claims 0.70	date of the contest that set the max rating
`repo_artifact`	repo exists, owner matches, not a fork, not README-only	0.45 + 0.05 × band index	0.70	languages ≥10% of bytes; frameworks 0.80 / 0.60	last commit
`claim_corroborated`	a skill claim is backed by a verified repo using that skill	0.70	0.80	the claimed skill (1.0)	newest backing repo's last commit
`cv_claim`	a skill / project / certification / achievement claim with no corroboration	0.30	0.20	the claimed skill(s) (1.0)	**the CV's own date**
`artifact_live`	a deployment responds	0.40	0.85	deployment 0.70, + linked repo's primary framework 0.60	probe date
`credential_verified`	the issuer confirmed a credential	0.60	0.95	credential / linked claim skills (1.0)	issue date
`claim_contradicted`	(defined in schema and parameters; **not emitted by v1.0.0**, see §15)	0.90	0.95	the claimed skill	—

5.4 Repository substance bands and strength

A repository's size and history put it in one of four ordinal bands. The bands are evaluated in order. Within a band, `_lt` conditions are OR-ed (any one is enough to demote) and `_gte` conditions are AND-ed (all must hold to promote). A repo matching nothing defaults to `substantial`.

band	index	condition	strength	observed value (6.3)
trivial	0	README-only, or commits < 10	0.45	0.35
small	1	commits < 50 **or** size < 100 kB	0.50	0.50
substantial	2	commits < 120	0.55	0.62
large	3	commits ≥ 120 **and** size ≥ 1000 kB	0.60	0.66

Notice that **the band produces two numbers in two different places**. The projection turns it into `strength` (how much signal), while 6.3 later turns the same band (carried in `payload.substance_band`) into an *observed value* (what ability it suggests).

5.5 Skill relevance from languages and frameworks

$$\text{relevance(language)} = \text{clamp}(0.5 + 0.5 \times \text{share}, 0.50, 0.95)$$

$$\text{kept only if share of bytes} \geq 0.10$$

A language below 10% of the bytes is *incidental* and is dropped entirely, because a CSS file in a TypeScript project is not evidence of CSS ability. Frameworks declared on the repo get 0.80, or 0.60 for secondary/datastore skills (`mongodb`, `jwt`, `redis`, `postgresql`). Names go through an alias table first (`mongoose→mongodb`, `next→nextjs`, `c++→cpp`, `golang→go`), and are lower-cased with spaces turned into underscores. That is how `Jupyter Notebook` becomes `jupyter_notebook`, which the ontology does not know.

Worked example: the `distributed-kv` repository (demo input)

168 commits, 1420 kB → **large** (index 3) → strength 0.45 + 0.05×3 = **0.60**.

Languages: Go 412 000 B, Shell 18 000 B, Dockerfile 4 200 B, total 434 200. Go's share is 0.949 → 0.5 + 0.5×0.949 = 0.974 → clamped to **0.95**. Shell (4.1%) and Dockerfile (1.0%) fall below 10% and are dropped.

Frameworks `docker`, `kubernetes` → **0.80** each.

GitHub ownership is `verified_oauth`, so no ownership cap applies: reliability = **0.70**.

Result: one `repo_artifact` event with skill_refs {docker 0.80, go 0.95, kubernetes 0.80}, dated 2026-06-10 (last commit).

5.6 Reliability factors: every discount is named

Reliability starts at the kind's `reliability_base` and is multiplied by named factors. Each applied factor is written onto the event (`reliability_factors[]`) with its value and reason, so a candidate can be told exactly why their evidence counted for less.

factor	value	applies to	direction scope
`ownership_unverified`	0.6	events attributed via an unproven GitHub account (repo, corroborated claim, linked deployment), never platform ratings	positive only

factor	value	applies to	direction scope
`scraped_source`	0.7	platform data read from a scraped page instead of an API	both
`aggregate_across_sources`	0.7	a claim M5 flagged as pooling counts over several platforms	positive only
`open_bound`	0.85	an open-ended claim (“500+” rather than “500”)	positive only
`stale_evidence`	1.0	nothing: deliberately inert, because decay handles staleness	—

Positive only encodes an asymmetry. A self-claimed account that *contradicts* the CV is credible, since nobody lists a handle that undermines them, while one that *corroborates* could belong to anyone. So the cap only applies to evidence that would raise a score.

Worked examples from the demo

CodeChef rating, scraped: 0.90 × 0.7 (`scraped_source`) = **0.63**.

Claim c24 “900+ problems across LeetCode, Codeforces, CodeChef”: a `cv_claim` at 0.20 × 0.7 (`aggregate_across_sources`) × 0.85 (`open_bound`) = **0.119**. Its payload also records `independently_confirmed`: Codeforces 734 and CodeChef 186. That verified part is already carried by the two `platform_rating` events, so only the unverifiable remainder is weighed here.

5.7 Dates: `occurred_at` is when it happened

The decay model reads `occurred_at`, so it must be when the *activity* happened, never when someone looked at it. Repos use the last commit, platforms the rating-defining contest, credentials the issue date. **Uncorroborated CV claims are dated at the CV document's own** `created_at`, not at verification time. Otherwise the weakest evidence in the system would get the freshest date and a two-year-old assertion would never decay. All timestamps are normalised to their UTC calendar day.

5.8 Deterministic event ids

```
event_id = "ev_" + sha256(verification_id | kind | discriminator)[:12]
```

The discriminator is what the event is *about* (e.g. the repo name, or `claim_id:skill`). The same report therefore always yields the same ids. That makes replays safe (6.2) and gives the fold a total order with `(occurred_at, event_id)`.

5.9 Suppressions and invariants

Anything not turned into an event is recorded in `not_emitted[]` with one of five reasons: `not_a_capability_signal` (forks, shells, unowned repos, dead deployments), `non_scoring_rule` (e.g. G3 unproven ownership, which is expressed as a factor instead), `no_skill_mapping` (employment, education, identifiers, an entrance rank), `institution_disclaimed` (CGPA or coursework the document itself disclaims), and `consent_scope_excluded`.

Before returning, `_check_invariants` enforces the contract and raises `ContractViolation` on any breach: every event has skills; ids are unique; reliability = base × product of factors (to 1e-6); all numbers are in [0, 1]; no event is in the future; every suppression reason is in the enum; and **every claim id appears either on an event or in a suppression**.

Demo result

30 claims and 7 repos become **21 events and 7 suppressions**. Two events name unknown skills (`haskell`, `jupyter_notebook`) and will be quarantined by 6.2.

6. 6.1 — Ontology: which skills exist and how they relate

The ontology answers three questions that every other submodule needs and none may decide for itself: *does this skill exist?* (`knows`), *how fast does it decay?* (`tier`), and *what does it imply?* (`relations`, `neighbours`). Everything comes from `contracts/ontology-seed.yaml`, version 0.1.0.

6.1 Contents of the seed

tier	skills (27 in total)
fundamental	algorithms, problem_solving, cpp, c, data_structures, cryptography, networking
ecosystem	typescript, javascript, python, java, go, rust, sql
framework	react, nextjs, nodejs, mongodb, postgresql, redis, jwt, deployment, aws, docker, kubernetes, numpy, pandas

from → to	type	γ	from → to	type	γ
cpp → algorithms	REQUIRES	0.65	typescript → javascript	PREREQUISITE_OF	0.55
cpp → c	RELATED_TO	0.40	nodejs → javascript	REQUIRES	0.65
algorithms → data_structures	RELATED_TO	0.40	postgresql → sql	REQUIRES	0.65
algorithms → problem_solving	PREREQUISITE_OF	0.55	kubernetes → docker	REQUIRES	0.65
nextjs → react	REQUIRES	0.65	numpy → python	REQUIRES	0.65
react → javascript	REQUIRES	0.65	pandas → numpy	REQUIRES	0.65

Edges are **directed**: evidence flows from source to target only. They are deliberately sparse, because an edge claims that evidence for one skill is genuine evidence for another.

6.2 Validation: a broken seed is refused

`SeedOntology.load()` is strict by default and `validate()` returns **every** problem, not just the first: an edge to a missing skill, a skill with no tier, an undeclared tier, a non-`lower_snake_case` id (which would split `React` and `react`), duplicated edges, self-edges, or a missing `ontology_version`. Cycles are allowed, because propagation is one hop from direct sources and cannot loop. `python -m m6_ontology check` exits non-zero on any problem, so it can gate CI.

6.3 Who uses which method

method	used by	for
`knows(skill_id)`	6.2	quarantine decisions
`ontology_version`	6.2, facade	stamped on every receipt and on `/versions`
`tier(skill_id)`	6.3	choosing λ, and therefore the half-life. Returns `None` rather than a default
`relations()` / `neighbours(id)`	6.3	propagation edges
`skill_ids()` / `skills()`	6.4	the `/skills` catalogue for filter UIs

7. 6.2 — Evidence ingestor: the gatekeeper and the archive

The plan's brief is exact: *validation, idempotency, consent refusal, immutable persistence, quarantine for unknown skills. No modelling.* The ingestor never computes a score. It only decides whether a batch may enter the log, and records what it did.

7.1 The ingest decision, step by step

1. **Validate** the batch against `evidence-event.schema.json` (M5 and M6 are separate services, so the schema is re-checked at the trust boundary), then check that no `event_id` repeats inside the batch. Failure: `SchemaInvalid`, HTTP 400, nothing written.
2. **Consent**: `consent_scope` must contain `skill_graph_inclusion`, otherwise `ConsentRefused`, HTTP 403. This is a decision, not a transient error, and retrying never succeeds.
3. **Take the exclusive lock** so that two concurrent deliveries cannot both decide the batch is new.
4. **Replay check**: if `batch_id` is known and its content hash matches, return `replayed=True` and write nothing. If the hash differs, raise `ConflictingEvent` (409).
5. **Per event**: if `event_id` is already stored with the same content, count it as a duplicate. With different content, raise 409, because history is not rewritten. Otherwise resolve each skill with `knows()`. Unknown skills mark the event as **quarantined for those skills only**.
6. **Append** the batch, events and `event_skill` rows in one transaction, and return the `Receipt`.

```
content_hash = sha256( json.dumps(doc, sort_keys=True, separators=(",", ":")) )
```

The hash ignores key order and whitespace, so two projections of the same report hash identically, while a projection that changed a strength does not.

7.2 Storage: three tables and four triggers

table	holds
`batch`	one row per batch: ids, subject, emitted_at, consent_scope, **not_emitted** (JSON), content_hash, event_count, ingested_at
`evidence_event`	one row per event: id, batch, subject, kind, occurred_at, strength, reliability, direction, content_hash, the full **document** (JSON)
`event_skill`	one row per (event, skill): relevance and `resolved` (0 = quarantined). Denormalised so 6.3 can fold one skill without parsing every document

Triggers `BEFORE UPDATE` and `BEFORE DELETE` on `evidence_event` and `event_skill` raise `'... is append-only'`. **Immutability is enforced by the database**, not by the absence of an update method, and the tests attack the database directly to prove it. The SQL avoids SQLite-only features, so a move to Postgres needs a driver swap and type-name changes rather than a rewrite.

7.3 Quarantine: held, not dropped

An event naming `python` and `jupyter_notebook` still counts for `python`. It is stored verbatim with `jupyter_notebook` flagged unresolved and held out of scoring for that skill only. Once the ontology adds the skill, the evidence is already there waiting. Dropping it would lose evidence; scoring it would invent a node nobody defined.

7.4 Concurrency

The HTTP server is threaded. Two bugs found by an eight-client race test were fixed. A shared SQLite connection fails under concurrent use, *reads included*, so all access is serialised on one re-entrant lock. And `ingest()` was check-then-act, so the whole decision is now held exclusively.

8. 6.3 — Scoring: how a score is computed

This is the mathematical core. In one sentence: **for each skill, start from a wide, pessimistic belief, then replay every relevant event in date order. Each event pulls the belief towards what it observed, by an amount set by how trustworthy it is, and time between events widens the belief again.** Everything below spells out that sentence.

8.1 The belief: a Gaussian in logit space

For each (candidate, skill), the state is `State(mu, var, at)`: a normal distribution with centre μ and variance σ² over *logit(ability)*, valid at date `at`.

Why logit space? Ability is shown on [0, 1], but a Gaussian update on a bounded quantity can walk outside the bounds and would then need clipping. Clipping biases estimates near the edges and quietly destroys the variance, which is the quantity the whole design rests on. logit maps (0, 1) to (−∞, +∞), so the Kalman update is exact there. Since sigmoid is monotone, a lower bound computed in logit space is still a true lower bound after transforming back.

p (display)	0.10	0.25	0.50	0.62	0.68	0.75	0.90
logit(p)	−2.197	−1.099	0.000	0.490	0.754	1.099	2.197

8.2 The prior: below median, and wide

$$\mu_0 = \text{logit}(\theta_0) = \text{logit}(0.25) = -1.0986 \qquad \sigma_0 = 1.30 \;\Rightarrow\; {\sigma_0}^2 = 1.69$$

A developer with no evidence is *unknown*, not average, so the prior is centred below the median. It is wide: ±1σ spans roughly 0.08–0.55 in display terms, so one weak event cannot manufacture a score. With no evidence at all, the displayed score is sigmoid(−1.0986 − 1.3) = **0.083**.

8.3 Step 1 of each event: the observed value

An event schema has no `observed` field. The projection only emits facts, and `ObservationModel.observed()` turns them into a display-space value using `m6-parameters.yaml → observed_value`, clamped to [0.01, 0.99]:

kind	method	observed value
`platform_rating`	percentile against reference	0.50 + 0.40 × ECDF_rated(max_rating)
`repo_artifact`	substance band	trivial 0.35 · small 0.50 · substantial 0.62 · large 0.66
`claim_corroborated`	constant	0.68 (the corroborating evidence, not the claimed band)
`artifact_live`	constant	0.58
`credential_verified`	constant	0.70
`cv_claim`	proficiency band	familiar 0.30 · intermediate 0.45 (default) · proficient 0.60 · expert 0.75
`claim_contradicted`	constant	0.15 (strong negative)

If an event's kind has no rule, or a band is unknown, the engine raises `UnscoreableEvent` instead of guessing. A silently defaulted observation would be a score nobody can explain.

The platform-rating ECDF

A raw rating (1687) cannot be used directly, because ratings on different platforms are not on the same scale. Instead the rating is converted to a percentile of the Codeforces rated population, using a lookup table with linear interpolation between anchors (`reference-distributions.yaml`). The reference population for algorithmic ability is developers generally, and being rated at all clears that floor. Hence the intercept a = 0.50 and slope b = 0.40.

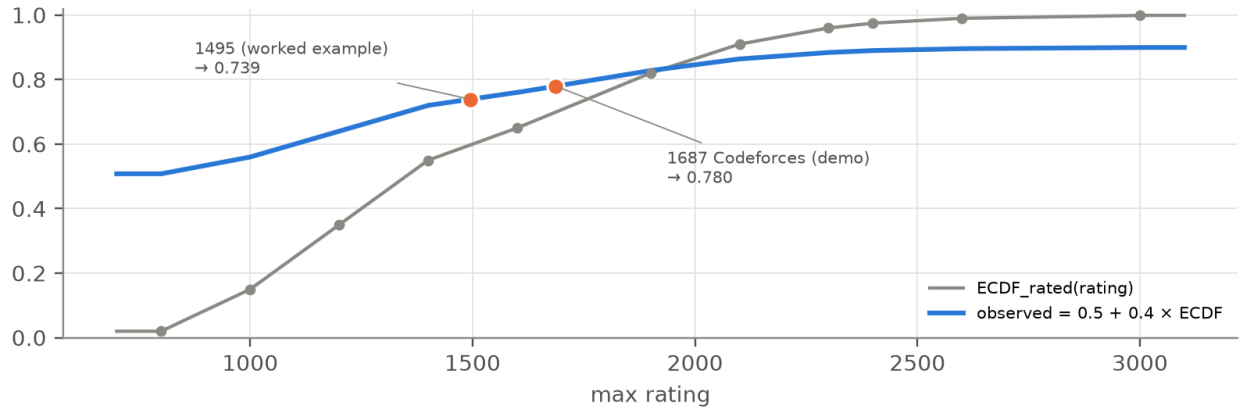


Figure 3 — Rating → percentile → observed value. Demo: Codeforces max 1687 lies between anchors 1600 (0.65) and 1900 (0.82): ECDF = 0.65 + (87/300)×0.17 = 0.699, observed = 0.50 + 0.40×0.699 = 0.780.

8.4 Step 2 of each event: the observation variance

$$\sigma^2_{obs} = S_0 \,/\, (\text{reliability} \times \text{relevance} \times \text{strength}), \qquad S_0 = 0.10$$

This is the single most important formula in the module. High reliability, relevance and strength give a small variance, which means a confident observation and a large update. S0 is the only free scale constant; everything else arrives on the event. (A guard of 1e-6 stops division by zero.)

> **Relevance is epistemic: it goes in the variance, never in the observed value**
>
> A repository that is 60% TypeScript is not evidence of *worse* TypeScript ability. It is *weaker evidence* about TypeScript ability. So all skills of one event share the same observed value, and only their variances differ. If relevance scaled the observed value instead, every incidental skill would be dragged toward zero. `test_observation.py` asserts this.

8.5 Step 3: the Kalman update

$$k = \sigma^2 \,/\, (\sigma^2 + \sigma^2_{obs})$$
$$\mu' = \mu + k \cdot (\text{logit}(\text{observed}) - \mu)$$
$$\sigma^{2\prime} = \sigma^2 \cdot (1 - k)$$

Intuition. *k*, the *gain*, is the fraction of the gap between belief and observation that is closed. It comes from comparing two uncertainties: when the current belief is vague (large σ²) and the observation is crisp (small σ²obs), k is close to 1 and the estimate jumps. When the belief is already tight, or the observation is noisy, k is small. Variance always shrinks (we always learn

something), and the new centre always lies between the old centre and the observation, so it never overshoots. Nothing is tuned: k falls out of the two variances.

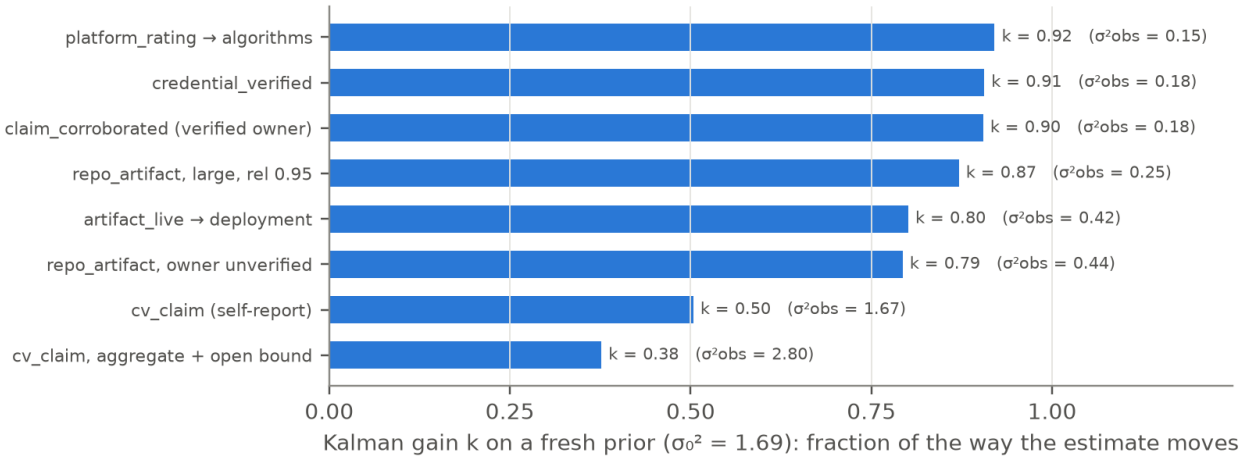


Figure 4 — How far a single event of each kind moves a fresh prior. An authoritative platform rating closes 92% of the gap. A bare CV claim closes 50% of a fresh prior's gap, but once real evidence has tightened the belief, the same claim closes only a few percent (python's cv_claim: k = 0.08).

The update in logit space treats direction symmetrically: negative evidence works because its observed value is low (0.15 for `claim_contradicted`), which pulls μ down. The `direction` field is carried for auditing and integrity checks, and is not a separate branch of the maths.

8.6 Step 4: the fold, recomputing from the log

```
def fold(events, skill, as_of):
    relevant = [e for e in events if e.occurred_at <= as_of and skill in e.skill_refs]
    relevant.sort(key=(occurred_at, event_id))       # total, deterministic order
    if not relevant:  return prior at as_of           # evidence_count = 0
    state = prior at relevant[0].occurred_at
    for e in relevant:
        state = decay(state, to=e.occurred_at, tier)   # time since last event
        observed, obs_var = observation_model(e, skill)
        state = kalman_update(state, observed, obs_var)
        record Contribution(e, gain, mean_before, mean_after)
    return decay(state, to=as_of, tier)               # time since newest event
```

Sequential Kalman updates depend on order, and events can arrive out of order (a 2024 repository verified in 2026). So the state is **never the source of truth**: it is always recomputed by folding the immutable log in `(occurred_at, event_id)` order. This one decision (O4) makes three things nearly free: `as_of` queries (a shorter fold), ablation (`fold_without(origin=…)`, a filtered fold) and parameter sweeps (a different `Parameters`, same fold). `rehydrate()` exists as a cheap read path that decays a cached state forward. A test asserts it agrees with a full fold to 1e-5, and it refuses a state stamped with a different `parameter_version`.

8.7 Step 5: from state to the numbers you see

$$\text{mean} = \text{sigmoid}(\mu) \qquad \text{score} = \text{sigmoid}(\mu - z\cdot\sigma) \qquad \text{upper} = \text{sigmoid}(\mu + z\cdot\sigma)$$

$$z = 1.0$$

$$\text{explain.sigma} = \text{mean} - \text{score} \quad \text{(display half-width)}$$

$$\text{explain.sigma_logit} = \sigma \quad \text{(the state)}$$

The interval is ±1σ in logit space mapped through the sigmoid, so in display space it is **asymmetric**. The `sigma` field in `/explain` is the display-space lower half-width, which is what reconciles the numbers in the original design documents. `sigma_logit` is the true state. z = 1 was *recovered* from the published worked example (mean 0.68, sigma 0.12, score 0.56), not chosen.

8.8 Worked example 1: `go`, step by step

The demo subject has two events naming `go`, both dated 2026-06-10: the `distributed-kv` repo (see §5.5) and claim c11 "go, proficient", which that repo corroborates. Query date: 2026-08-30. Tier: ecosystem ($\lambda = 3\times10^{-4}$/day). Every number below was computed by hand and matches the engine's output.

step	computation	μ (logit)	σ²	display
prior	μ0 = logit(0.25), σ0² = 1.3²	−1.0986	1.6900	mean 0.250, [0.083, 0.550]
event 1 `repo_artifact` (ev_163d…)	observed 0.66 (large) → logit 0.6633. σ²obs = 0.10/(0.70×0.95×0.60) = 0.2506. k = 1.69/(1.69+0.2506) = **0.871**. μ = −1.0986 + 0.871×(0.6633+1.0986)	0.4357	0.2183	mean 0.607
event 2 `claim_corroborated` (ev_aa67…), same day, no decay	observed 0.68 → logit 0.7538. σ²obs = 0.10/(0.80×1.0×0.70) = 0.1786. k = 0.2183/(0.2183+0.1786) = **0.550**. μ = 0.4357 + 0.550×(0.7538−0.4357)	0.6107	0.0982	mean 0.648
decay 81 days to as_of	σ² += 0.0003×81 = 0.0243. pull = 1 − e^(−81×0.0003/0.35) = 0.0671. μ += (−1.0986 − 0.6107)×0.0671	0.4961	0.1225	σ = 0.350
result	score = sigmoid(0.4961 − 0.350), mean = sigmoid(0.4961), upper = sigmoid(0.4961 + 0.350)			**[0.536, 0.700]**, mean **0.622**

Same-day events are ordered by `event_id`: `ev_163d…` comes before `ev_aa67…`, so the repo is applied first. The explain response reports exactly these weights (0.871, 0.550), `sigma` = 0.622 − 0.536 = 0.085 and `sigma_logit` = 0.35. Notice that **the first event carries the most weight**: against a wide prior the first observation moves the estimate furthest.

8.9 Worked example 2: `python`, and why a weak claim barely matters

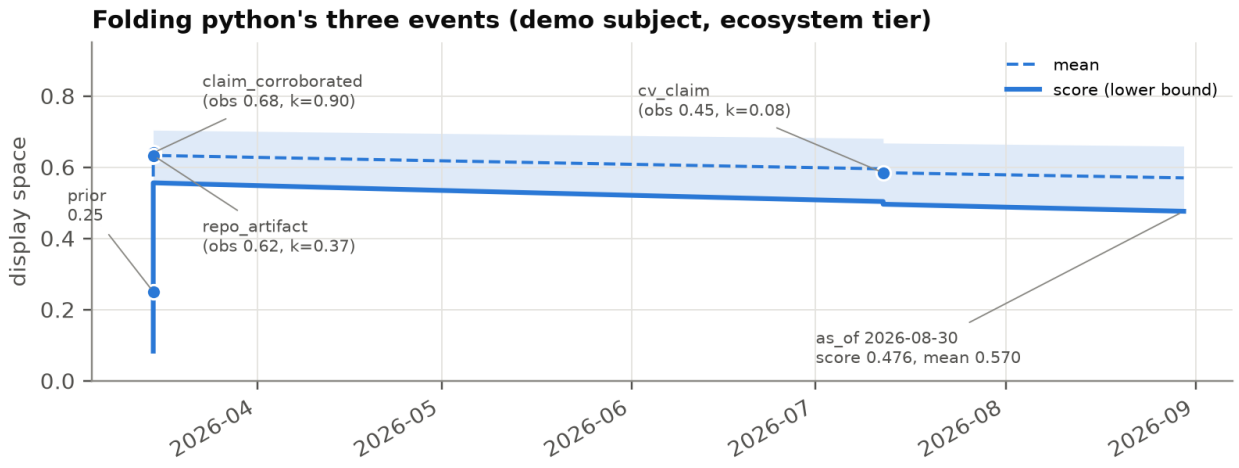


Figure 5 — The belief for `python` through its fold. The band (shaded) is the interval; the solid line is the published score. Between events the band slowly widens and the mean drifts down: that is decay.

event	observed	relevance	σ²obs	gain k	mean after
claim c12 “python, expert” corroborated (2026-03-15)	0.68	1.00	0.179	0.904	0.640
`ml-pipeline` repo, substantial (97 commits), same day	0.62	0.927	0.280	0.366	0.633
decay 119 days → 2026-07-12					0.595
`cv_claim` c10: project `thesis-notebooks` (repo does not exist)	0.45	1.00	1.667	0.077	0.584
decay 49 days → as_of 2026-08-30					**0.570**, score 0.476

The `cv_claim` has reliability 0.20 and strength 0.30, so its variance (1.667) is about nine times the corroborated claim's. Arriving after real evidence has tightened the belief, it gets a gain of only 0.077 and moves the mean by 0.011. **A weak event next to strong ones barely moves the posterior**, which is the behaviour the design wants.

8.10 Worked example 3: `rust`, a lone self-report

Rust's only event is claim c19 “rust, familiar”, which nothing corroborates: observed 0.30, σ^2obs = 1.667, against the fresh prior (σ^2 = 1.69). k = 0.504, so the mean only moves from 0.250 to 0.274, while the variance halves to about 0.84. After 49 days of decay the result is **[0.130, 0.487]**, the widest and lowest interval in the graph. Low *reliability*, not a low observed value, is what stops an uncorroborated claim from producing a score.

9. Decay: how scores fade when evidence gets old

The design principle (O6): **the score falls because we know less, not because the person got worse.** Ability does not vanish when someone stops pushing to GitHub, but our *evidence* about it does age. Decay therefore does two things whenever time passes, both in `Filter.decayed_to(state, when, tier)`:

$$\Delta t = \text{days between state.at and when} \qquad (\Delta t \le 0 \Rightarrow \text{no change})$$

$$\sigma^2(t) = \sigma^2(\tau) + \lambda_s \cdot \Delta t \qquad \text{variance grows linearly}$$

$$\mu(t) = \mu(\tau) + (\mu_0 - \mu(\tau)) \cdot (1 - e\text{^}(-\Delta t \cdot \lambda_s / k)) \qquad \text{centre reverts to the prior}$$

$$h_s = k / \lambda_s , \qquad k = 0.35$$

9.1 The two effects

- **Variance growth** ($\sigma^2 + \lambda\Delta t$). This is a random-walk model: the longer since we last observed the skill, the more it could have changed in either direction. The interval widens on *both* sides, and since the published score is the lower bound, the score falls.
- **Mean reversion** ($\mu \to \mu 0$). The centre drifts exponentially back toward the prior 0.25, with time constant h_s. After a very long silence we are back to *"unknown"*, which by design is below median.
- Both depend on the skill's **tier**, which sets λ. Mean reversion is not a separate knob: h_s = k/λ_s ties it to the same volatility, so there is one swept constant (k) instead of three. A test checks that *most of the score lost is uncertainty, not a downgrade*.

9.2 The three tiers

tier	λ per day	sweep range	h = 0.35/λ	true half-life of μ (h·ln2)	σ² added per year
fundamental	1.0×10^{-4}	3×10^{-5} – 3×10^{-4}	3500 d (9.6 y)	2426 d (6.6 y)	0.037
ecosystem	3.0×10^{-4}	1×10^{-4} – 8×10^{-4}	1167 d (3.2 y)	809 d (2.2 y)	0.110
framework	7.5×10^{-4}	3×10^{-4} – 2×10^{-3}	467 d (1.3 y)	323 d (0.9 y)	0.274

Tiers reflect how fast an area's release cadence makes old evidence stale: reasoning about algorithms in 2019 still transfers, while React knowledge from 2019 is partly obsolete. A tier is a claim about the **skill**, never about a candidate. Skills the ontology does not tier fall back to `m6-parameters.yaml`'s list, then to `ecosystem`.

> **A precise note on the word "half-life"**
>
> The code and the explain response call h = k/λ the `half_life_days`. Strictly, h is the exponential **time constant**: after h days the mean has covered $1 - e^{-1} \approx$ **63%** of the distance back to the prior, not 50%. The true half-life of the mean's gap is h·ln 2 (table above). This does not affect any computed score. It matters only when you describe the numbers in words.

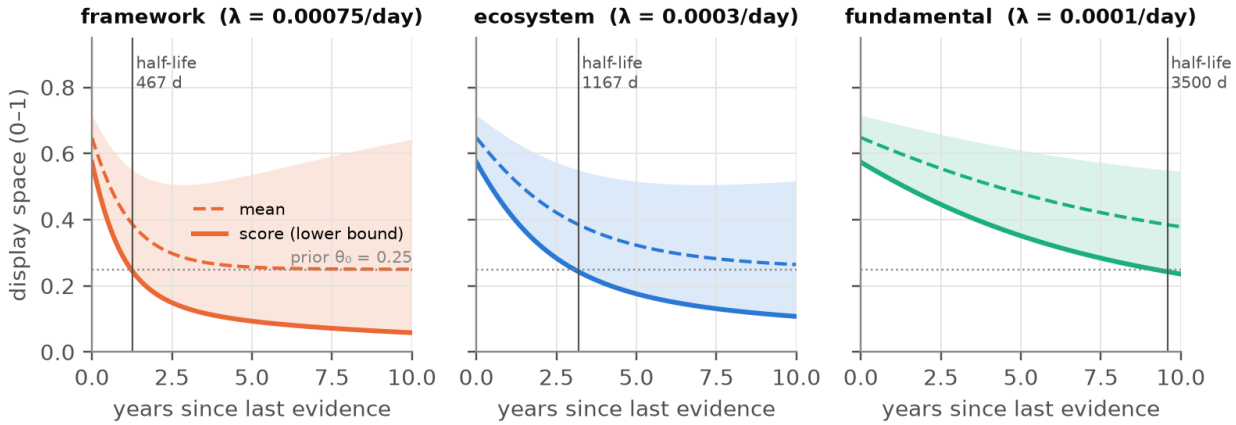


Figure 6 — The same starting belief (go just after its two updates: mean 0.648) left with no new evidence for ten years under each tier. Dashed = mean, solid = published score, band = interval. A framework skill loses its score within about two years; a fundamental skill holds for most of a decade. In the framework panel the upper bound eventually rises: we are no longer confident about anything, in either direction.

9.3 Worked decay examples

- **go, 81 days (ecosystem).** σ^2 0.0982 → 0.1225 and μ 0.6107 → 0.4961 (§8.8). The mean falls 0.648 → 0.622, and the score falls by more because the interval also widened.
- **The whole demo graph two years later**, with no new evidence (demo step 4):

skill	tier	2026-08-30	2028-08-30
algorithms	fundamental	`[0.703, 0.811]`	`[0.582, 0.757]`
problem_solving	fundamental	`[0.692, 0.804]`	`[0.572, 0.750]`
cpp	fundamental	`[0.556, 0.708]`	`[0.452, 0.721]`
go	ecosystem	`[0.536, 0.700]`	`[0.346, 0.636]`

go (ecosystem) loses 0.19 of score in two years; algorithms (fundamental) loses 0.12. For cpp the upper bound actually went up (0.708 → 0.721): the interval widens on both sides.

9.4 Where decay happens

- **Inside the fold, between events** (the gap from the previous event to this one).
- **At the end of the fold**, from the newest event to `as_of`. This is why the same log gives different answers for different `as_of` dates.
- **In propagation**, where the source state is decayed to `as_of` before it is used.
- **Never at ingest.** The `stale_evidence` reliability factor is deliberately 1.0, because discounting reliability for age as well would count staleness twice.

Integrity check `WHOLLY_STALE` (6.5) uses each skill's own h: the same 20-month-old evidence is stale for a framework and fresh for a fundamental. In the demo, `aws` (a credential from 526 days ago, framework tier, h = 467 d) is flagged.

10. Propagation: scores for skills with no direct evidence

If someone demonstrably writes good C++, that says something, though less, about C. Propagation lets a skill with **no direct evidence** borrow a damped estimate from a neighbour across an ontology edge. Its rules (O5, O7) are strict:

- **Read time only.** Nothing propagated is ever stored. A stored propagated state could become a source for further propagation, and evidence would drift arbitrarily far.
- **One hop.** `depth: 1` in the contract.
- **Direct-evidence sources only.** A source must have `evidence_count > 0`. This is what makes the pass non-iterative and loop-free without cycle detection.
- **Direct evidence always wins.** A target with its own evidence is never overwritten.
- **One γ per edge type, never per edge.** REQUIRES 0.65 · PREREQUISITE_OF 0.55 · RELATED_TO 0.40.

10.1 The mechanism: damping is variance inflation

near = source state decayed to as_of far = target's prior at as_of

$$\sigma^2_{obs} = \sigma^2_{near} / \gamma \qquad k = \sigma^2_{far} / (\sigma^2_{far} + \sigma^2_{obs})$$

$$\mu' = \mu_{far} + k \cdot (\mu_{near} - \mu_{far}) \qquad \sigma^{2\prime} = \sigma^2_{far} \cdot (1 - k)$$

The neighbour's belief enters as a *pseudo-observation* whose variance is inflated by 1/γ, through the same Kalman update used for real events. A weak edge is simply a noisy observation, so weak evidence and distant evidence are treated the same way. If several edges reach the same target, they are applied one after another, each on the result of the previous one.

Worked example: `c` inferred from `cpp` (demo)

cpp at as_of: μ = 0.5556, σ² = 0.108 (three direct events). Edge cpp → c is RELATED_TO, γ = 0.40. σ²obs = 0.108 / 0.40 = 0.270. Target prior: μ = −1.0986, σ² = 1.69. k = 1.69/(1.69+0.27) = 0.862. μ = −1.0986 + 0.862×(0.5556 + 1.0986) = 0.3277; σ² = 1.69×0.138 = 0.2328, σ = 0.4825. Display: mean 0.581, interval **[0.461, 0.692]**. The interval is wider than cpp's [0.556, 0.708], as it should be.

10.2 The presentation problem, and how it is handled

A propagated skill can outrank skills with real evidence. In the golden report `c` (0.370, inferred) sits above `python` (0.189, one direct event). The design response is to make this *visible*, not to clamp the number. Propagated states carry `evidence_count: 0`, `prior_only: true`, `basis: "propagated"` and a `propagated` block naming the source, edge type and γ. 6.4 labels them *inferred*, search excludes them unless `include_inferred=true`, and 6.5 raises `INFERRED_ONLY`.

11. 6.4 — Query API: skills, explanations and search

11.1 Endpoints

endpoint	returns	parameters
`GET /health`	status, number of subjects, parameter_version	—
`GET /skills`	ontology catalogue with tier and half-life per skill	—
`GET /developers/{ref}/skills`	the graph: one row per skill with `score`, `interval`, `mean`, `volatility_tier`, `evidence_count`, `inferred`, `days_since_newest_evidence`, `propagated`	`as_of`, `propagate`, `include_inferred`, `min_evidence`
`GET /developers/{ref}/skills/{skill}/explain`	one skill decomposed (below)	`as_of`, `propagate`
`GET /search?skill=…`	tiers of candidates	`min`, `definite` (default true), `include_inferred` (default false), `as_of`
`GET /developers/{ref}/score` (also `rating`, `trust`, `overall`)	**400**: “there is no overall score…”	—

Every endpoint takes `as_of=YYYY-MM-DD` (default: today). The rows are sorted by lower bound, but the interval always travels with the row.

11.2 The explain response (the demo endpoint)

field	meaning
`score`, `interval`, `mean`	lower bound, [lower, upper], centre (display)
`sigma`, `sigma_logit`	display half-width (mean − score); logit-space σ
`tier`, `half_life_days`	decay tier and h = k/λ
`evidence_count`, `prior_only`, `basis`	how many events; whether the number is just the prior; `direct_evidence` / `propagated` / `prior`
`days_since_newest_evidence`	the age that drives the final decay
`contributions[]`	per event in fold order: `event_id`, `kind`, `occurred_at`, `observed`, `relevance`, `weight` (the Kalman gain), `moved_mean_to`
`propagated`	`{from_skill_id, edge_type, gamma}` or null
`not_counted[]`	every suppression from this subject's batches, plus quarantined events naming this skill
`parameter_version`, `subject_ref`, `as_of`	provenance

`not_counted` lets a candidate see that a claim was *read and rejected* rather than never noticed, so “silence is never an option” reaches all the way to the user.

11.3 Search: tiers, never a ranking

`meets_threshold` has two readings, and the caller must choose: `definite=true` means the **whole interval** clears the bar (lower ≥ min: confidently above), while `definite=false` means the interval **reaches** it (upper ≥ min: might be above). This is a recruiter's risk decision, so the API makes it explicit.

`tiers()` sorts the matches by lower bound (descending) and walks down them, tracking the lowest lower bound (the *floor*) of the current tier. A candidate starts a new tier only if its **upper** bound is below the current floor, meaning it is definitely below every member of the current tier. Inside a tier, members are listed **alphabetically**, because ordering them by point score would recreate the false precision the tiers exist to prevent.

```
A [0.60, 0.80]  B [0.55, 0.75]  C [0.30, 0.50]  D [0.20, 0.58]
tier 1: A        floor 0.60
B: upper 0.75 >= 0.60 -> joins tier 1,  floor 0.55
C: upper 0.50 <  0.55 -> starts tier 2, floor 0.30
D: upper 0.58 >= 0.30 -> joins tier 2
=> tier 1 {A, B}   tier 2 {C, D}
```

A measured finding: at z = 1, search filters rather than ranks

The intervals are wide relative to real differences. A candidate whose repositories are all *trivial* rather than *substantial* (about 0.10 of mean ability) still overlaps at z = 1, and the pair only separates at z ≤ 0.40. Lower reliability alone barely moves the mean at all: reliability controls *confidence*, not *location*. If the product needs search to rank, the lever is `display.z` (swept 0.5–2.0), at the cost of false precision. Two tests pin this behaviour.

12. 6.5 — Integrity analytics: what a careful reader should notice

The obvious design would discount suspicious evidence inside 6.3. That is exactly what this module must not do: a signal that silently moved a score would be an unexplainable penalty. So 6.5 produces **findings** beside the graph, and `test_neutrality.py` asserts that the graph is byte-identical before and after the analyser runs. Three properties are structural: each finding **names its evidence** (event ids, skills), **states the innocent explanation**, and has only two severities, `notice` and `attention`. There is no “fail”.

check	severity	fires when	threshold
`EVIDENCE_BURST`	attention	most evidence is from just before verification	≥60% of ≥4 events within 45 days
`OWNERSHIP_UNPROVEN`	attention	much of the graph rests on an unproven account	≥50% of ≥3 events carry `ownership_unverified`
`CONTRADICTED`	attention	a skill has both positive and negative events	any
`SINGLE_SOURCE`	notice	every event for a skill has one origin	≥2 events
`CLAIM_ONLY`	notice	a skill rests only on self-report	all `cv_claim`
`INFERRED_ONLY`	notice	a skill's score is propagated only	evidence_count 0 + propagated
`WHOLLY_STALE`	notice	every event is older than the skill's own h	per-tier h
`BREADTH_FROM_ONE_ARTEFACT`	notice	one event yields many skills	≥5 skills
`QUARANTINED_EVIDENCE`	notice	evidence held out over unknown skills	any

Findings are ordered attention-first, then by how much of the graph they touch, then by id, and are capped at 40. **Demo result:** 12 findings, all `notice`: breadth from one artefact (the ml-pipeline repo, 5 skills), quarantined evidence (haskell, jupyter_notebook), claim-only (haskell, rust), inferred-only (c, data_structures, javascript, sql), single-source (go, postgresql, typescript), and wholly-stale (aws). Because this candidate's GitHub is OAuth-verified, `OWNERSHIP_UNPROVEN` stays quiet; on the golden report, where ownership is unproven, it fires.

13. How to run it and get the output

13.1 Prerequisites

- **Python 3.12** (developed and verified on 3.12.3).
- **PyYAML** and **jsonschema**: the only third-party imports. Everything else is the standard library (`sqlite3`, `http.server`, `hashlib`, …). **pytest is not required.**
- `make` (optional; every target is a plain command you can type yourself).
- Every command needs `PYTHONPATH=packages`. The Makefile sets this for you.

```
cd ~/BTP-ABC3
python3 -c "import yaml, jsonschema"      # if this fails: pip install pyyaml jsonschema
make help                                  # list every target
```

13.2 The fastest path: three commands

```
make test        # 562 tests, every package               -> "562 passed, 0 failed"
make demo        # the narrated 9-step walkthrough         -> writes demo/output/*.json
make all         # test + ontology check + verify + recovery gate + demo
```

13.3 All Make targets

target	what it does
`make test [FILTER=x]`	run all tests (or those whose name contains x)
`make demo` / `make demo-http`	the walkthrough / plus every HTTP endpoint live on an ephemeral port
`make ontology`	6.1: validate the seed, list skills and edges
`make project`	6.0: project the golden report `docs/docs/examples/report-full.json` → `batch.json`
`make batch`	6.0: project both the full and the degraded example report
`make ingest`	6.2: project, then ingest `batch.json` into `evidence.db`
`make score [AS_OF=…]`	6.3: score the whole graph from `evidence.db` (default as-of 2026-08-24)
`make query`	6.4: the graph as the API presents it
`make why SKILL=cpp`	6.4: the explain response for one skill
`make find SKILL=typescript MIN=0.3`	6.4: tiered search
`make integrity` / `make profile`	6.5 findings / skills + integrity for the golden subject
`make serve` / `make api`	ingestor on :8100 / query API on :8200 (both on `evidence.db`)
`make recover`	6.3 parameter-recovery gate (exit 1 on failure)
`make sweep`	list every swept parameter with its range
`make verify`	re-check `m6_model.py` against the worked example
`make clean`	remove `__pycache__`, `batch*.json`, `evidence.db`

Note that `make ingest`, `score`, `query`, `why` and `find` each depend on the previous step, so a single `make why SKILL=cpp` re-projects and re-ingests first. Re-ingesting is a harmless no-op.

13.4 Step by step, one submodule at a time

This is what happens on the golden report (`report-full.json`, subject `subj_7a3f9e21`), with real output:

```
$ export PYTHONPATH=packages
$ python3 -m m6_projection docs/docs/examples/report-full.json --out batch.json
8 events, 4 suppressed, 0 quarantined
  not_emitted[not_a_capability_signal] W1 — https://cf-connect-rs.vercel.app returns 404
  not_emitted[no_skill_mapping] employment, education, identifier and profile claims — ...
  not_emitted[institution_disclaimed] claims disclaimed by the document itself: coursework
  not_emitted[non_scoring_rule] G3 — GitHub ownership unproven; expressed as a reliability factor

$ python3 -m m6_ingestor ingest batch.json --db evidence.db
8 accepted, 0 duplicate, 0 quarantined
log now holds 8 events

$ python3 -m m6_scoring score --db evidence.db --as-of 2026-08-24
skill               score   mean   band  evid  source
  algorithms        0.532  0.638  0.106     1  1 direct
  problem_solving   0.519  0.626  0.107     2  2 direct
  cpp               0.507  0.620  0.114     2  2 direct
  data_structures   0.397  0.549  0.152     0  propagated from algorithms
  typescript        0.381  0.504  0.123     3  3 direct
  c                 0.370  0.526  0.157     0  propagated from cpp
  ...
  python            0.189  0.339  0.150     1  1 direct

$ python3 -m m6_query explain --db evidence.db --skill typescript --as-of 2026-08-24
$ python3 -m m6_query search  --db evidence.db --skill typescript --min 0.3 --as-of 2026-08-24
$ python3 -m m6_scoring recover          # the gate: PASS / FAIL
$ python3 -m m6_ontology show cpp        # tier + both directions of propagation
```

13.5 Running the HTTP services

```
# terminal 1 - the ingestor (write side)
make serve                                  # http://127.0.0.1:8100
# terminal 2 - push a batch
curl -X POST localhost:8100/evidence/batch -H 'Content-Type: application/json' -d @batch.json
curl localhost:8100/quarantine

# the query API (read side) on the same database
make api                                    # http://127.0.0.1:8200
curl 'localhost:8200/developers/subj_7a3f9e21/skills?as_of=2026-08-24'
curl 'localhost:8200/developers/subj_7a3f9e21/skills/typescript/explain?as_of=2026-08-24'
curl 'localhost:8200/search?skill=typescript&min=0.3&definite=false'
curl -i localhost:8200/developers/subj_7a3f9e21/score     # 400, by design
```

In deployment 6.2 and 6.4 are separate services, because writing and reading scale differently. To run **all of M6 as one process**, including `POST /evidence/report` (a raw M5 report, projected and ingested in one call), `/versions`, `/integrity` and `/profile`:

```
PYTHONPATH=packages python3 -c "
from m6 import M6; from m6.api import serve
m6 = M6('evidence.db'); print('listening on :8300'); serve(m6, port=8300).serve_forever()"

curl -X POST localhost:8300/evidence/report -d @demo/input/verification-report-extensive.json
curl 'localhost:8300/developers/subj_c4f80a12/profile?as_of=2026-08-30'
```

13.6 Using it from Python

```
import json
from datetime import date
from m6 import M6

report = json.load(open("demo/input/verification-report-extensive.json"))
with M6("evidence.db") as m6:                       # ":memory:" for a throwaway run
```

```
receipt = m6.ingest_report(report)              # 6.0 then 6.2
print(receipt.to_dict())
as_of = date(2026, 8, 30)
for row in m6.skills("subj_c4f80a12", as_of):   # 6.3 + 6.4
    print(row.skill_id, row.to_dict()["interval"])
print(m6.explain("subj_c4f80a12", "go", as_of)) # one skill decomposed
print(m6.integrity("subj_c4f80a12", as_of).to_dict())
print(m6.versions())                            # every contract version behind the answer
```

13.7 Running on your own report

```
python3 demo/run_demo.py --input path/to/report.json --as-of 2026-10-24
python3 demo/run_demo.py --step 5 --skill cpp      # just the explain step, for cpp
```

The report must validate against `docs/docs/schemas/verification-report.schema.json` and must carry `subject.consent_ref`. Without it you get zero events, which is correct behaviour.

13.8 Reading the demo output

step	what you see	what to look for
1 input	report summary: 30 claims, 7 repos, 2 platforms, …	the CV is dated 6 weeks before verification
2 projection	21 events and 7 suppressions, with named discounts	CodeChef at 0.63 (scraped); CV claims dated 2026-07-12; the 900+ claim at 0.119
3 ingestion	21 accepted, 2 quarantined; replay → 0 accepted; consent refused; UPDATE refused	haskell and jupyter_notebook are stored, not dropped
4 scoring	19 skills as intervals, then the same graph in 2028	four inferred skills; every interval wider two years on
5 explain	go: weights 0.871 and 0.550; what was not counted	the first event weighs most
6 search	go ≥ 0.40, definite vs reaches	tiers, not ranks
7 integrity	12 notice findings, each with its innocent explanation	no attention findings
8 guarantees	/score → 400; graph identical after 6.5; all versions	
9 HTTP	(with `--http`) every endpoint over real sockets	

The files in `demo/output/` are the machine-readable versions: `01-projected-batch.json` (the batch exactly as 6.2 receives it), `02-ingest-receipt.json`, `03-skill-graph.json`, `04-explain.json` and `05-integrity.json`.

14. How we know it works

14.1 The test suite

`make test` runs **562 tests with 0 failures** (measured while writing this guide) via the zero-dependency runner. Each package has its own folder. Some tests are worth knowing about because they protect design guarantees rather than individual functions:

- **Reference parity** (`test_reference_parity.py`): prior, update, decay, variance, fold, explain and propagation all match `contracts/reference/m6_model.py` to 1e-9.
- **Golden test** (`test_golden.py`): pins the projected batch to the published trace and reproduces the skill graph.
- **Invariants provoked** (`test_invariants.py`): each of the six projection invariants is triggered on purpose, with control tests so none can pass vacuously.
- **Mutation-style integration** (`test_integration.py` in 6.1): make the ontology lie and check that something notices, including that the *tier* really changes the score.
- **Triggers attacked directly** (`test_persistence.py`): UPDATE/DELETE against SQLite must fail.
- **Neutrality** (`test_neutrality.py`): 6.5 moves no number.
- **Constraints** (`test_constraints.py`): no response contains an aggregate-shaped key, and no public method is named like one.
- **Facade agreement**: calling through `M6` gives byte-identical results to calling the submodules directly.

14.2 The parameter-recovery gate

"If the filter cannot recover a known ability from synthetic evidence, nothing downstream is worth building." `make recover` invents 300 developers with known true abilities (spread evenly from 0.10 to 0.90), generates 12 noisy events each through the same variance formula the engine uses, folds them with the real filter, and compares:

```
parameter recovery — 300 trials x 12 events
  bias      -0.0236   (want |bias| < 0.05)
  rmse       0.0510   (want < 0.15)
  coverage   0.677    (truth inside mu +/- z*sigma; want 0.45-0.95)
PASS
rmse vs events:  1 -> 0.136   2 -> 0.089   4 -> 0.076   8 -> 0.054   16 -> 0.044   32 -> 0.035
```

Coverage of 0.677 against the nominal 0.683 for ±1σ means the filter's stated uncertainty matches its actual error. The slightly negative bias comes from the deliberately below-median prior, which is the safe direction to be wrong in. The RNG is seeded, so a regression shows up as a changed number, not a flaky test.

14.3 The sensitivity sweep

`make sweep` lists the 11 swept constants (`prior__theta_0`, `prior__sigma_0`, `observation__S0`, three `decay__lambda_per_day__*`, `decay__h_ratio_k`, three `propagation__gamma_per_edge_type__*`, `display__z`). They are *discovered* by walking the YAML, so adding a `sweep:` range to the contract is enough to include a new one. `Parameters.replacing(prior__theta_0=0.3)` gives a modified copy without touching the file, and `ScoringEngine.fold_without(origin="github_api")` gives per-source ablation.

15. Caveats and gotchas

Things found while preparing this guide, or recorded in the code itself, that are worth knowing before you rely on any number or present the module:

1. **The Codeforces ECDF is approximate.** `reference-distributions.yaml` says so in its `status` field (`approximated_pending_regeneration`). Its anchors are estimated from the rank boundaries, not computed from `/api/user.ratedList`. `platform_rating` is the strongest event in the system, so regenerate this table before publishing results.
2. `claim_contradicted` **is never emitted.** The schema, `projection-rules.yaml` and `m6-parameters.yaml` define it, and 6.3 and 6.5 would handle it, but `projector.py` (v1.0.0) has no branch that produces it. The demo's "Rust claim with no Rust anywhere" discrepancy becomes an ordinary `cv_claim`. As a result `CONTRADICTED` cannot fire on real input today.
3. **"half-life" is a time constant.** h = k/λ is the e-folding time of the mean reversion; the true half-life is h·ln 2 (see chapter 9).
4. **Search tiers are justified by one pair, not every pair.** A new tier starts when a candidate is below the current tier's floor, but a later candidate can still overlap a member of an earlier tier (D [0.20, 0.58] vs B [0.55, 0.75] in the example in §11.3). Treat a tier boundary as "at least one member definitely separates", not "every pair across it is separated".
5. **Docstrings say 410, the code returns 400** for `/developers/{ref}/score`. The READMEs and tests agree on 400.
6. `make explain` **does not exist.** The scoring README mentions it; the real target is `make why SKILL=…`.
7. **SQLite, not Postgres.** The SQL is kept portable, but the agreed production database is Postgres.
8. **M5-only producer.** The event schema's `source.module` is the constant `m5-cv-verification`, and the `kind` enum has no `assessment_result`. When M4 (assessments) starts feeding the graph, the fix is additive: widen those two fields and add priors and an observed-value rule to `m6-parameters.yaml` (`context_difficulty` is already reserved for this).

16. Appendix: formula sheet and parameter reference

16.1 Every formula in one place

what	formula	where
repo strength	0.45 + 0.05 × band_index	rules.repo_strength
language relevance	clamp(0.5 + 0.5 × share, 0.50, 0.95), dropped if share < 0.10	rules.language_relevances
reliability	reliability_base × Π factor values (direction-scoped)	rules.apply_factors
event id	"ev_" + sha256(vid|kind|disc)[:12]	rules.event_id
platform observed	0.50 + 0.40 × ECDF(max_rating)	observation._from_percentile
observation variance	S0 / (reliability × relevance × strength)	observation.variance
prior	μ0 = logit(0.25), σ0² = 1.69	filter.initial
gain	k = σ² / (σ² + σ²obs)	filter.gain
update	μ += k (logit(obs) − μ); σ² ×= (1 − k)	filter.updated
decay: variance	σ² += λ_tier × Δt	filter.decayed_to
decay: mean	μ += (μ0 − μ)(1 − e^(−Δt λ / k)), k = 0.35	filter.decayed_to
half-life (time constant)	h = 0.35 / λ_tier	params.half_life
propagation	σ²obs = σ²_source / γ_type, then the same update on the target's prior	propagation._hop
display	score = sigmoid(μ − zσ), upper = sigmoid(μ + zσ), mean = sigmoid(μ), z = 1	filter.State
tiering	new tier iff candidate.upper < current tier's lowest lower bound	ranking.tiers

16.2 m6-parameters.yaml, v1.0.0

parameter	value	sweep	meaning
`prior.theta_0`	0.25	0.15 – 0.40	prior ability (display)
`prior.sigma_0`	1.30	0.9 – 2.0	prior spread (logit)
`observation.S0`	0.10	0.05 – 0.30	observation-variance scale
`observation.context_difficulty`	1.0	—	reserved for M4 assessments
`decay.lambda_per_day.fundamental`	1.0e-4	3e-5 – 3e-4	variance growth per day
`decay.lambda_per_day.ecosystem`	3.0e-4	1e-4 – 8e-4	
`decay.lambda_per_day.framework`	7.5e-4	3e-4 – 2e-3	
`decay.h_ratio_k`	0.35	0.15 – 0.70	h = k/λ
`propagation.gamma.PREREQUISITE_OF`	0.55	0.3 – 0.8	edge damping
`propagation.gamma.REQUIRES`	0.65	0.4 – 0.9	
`propagation.gamma.RELATED_TO`	0.40	0.2 – 0.6	
`propagation.depth`	1	—	hops
`display.z`	1.0	0.5 – 2.0	interval half-width in σ

16.3 Projection priors (projection-rules.yaml)

kind	strength	reliability_base
platform_rating	0.80	0.90
repo_artifact	0.45–0.60 by band	0.70
claim_corroborated	0.70	0.80
artifact_live	0.40	0.85
credential_verified	0.60	0.95
cv_claim	0.30	0.20
claim_contradicted	0.90	0.95

16.4 Where to read more

- `docs/docs/m6-decisions.html`: the seven modelling decisions O1–O7, each with the alternative that was rejected.
- `docs/docs/m6-worked-trace.html`: the golden report traced end to end.
- `docs/docs/m6-skill-graph-plan.html`: the original plan and week-by-week scope.
- `packages/*/README.md`: one per submodule, with design notes and test inventories.
- `docs/reports/module6-skill-graph-report.pdf`, `module6-contracts-reference.pdf`: the formal reports.